PukSquash Developer Manual

User Profile Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Profile management lets a signed-in user edit the name shown in the application and on bookings. The Firebase edition keeps authentication credentials separate from the relational profile. User, Coach and Admin accounts use the same profile page.

UserProfile loads refreshAccount on mount, splits full_name into first and remaining names, and displays account details. Edit enables the name fields. Save opens a confirmation dialog, updateAccount sends PATCH /api/me, and the successful response replaces the shared profile. Cancel restores the last saved account values.

1 1 Files and Responsibilities

resources/js/pages/UserProfile.vue — editable form and confirmation

resources/js/composables/useAccount.js — shared profile and updateAccount

app/Http/Controllers/Api/UserController.php — update validation

app/Repositories/UserRepository.php — UpdateUserProfile and cache invalidation

1 2 API Contract

GET /api/me returns the profile. PATCH /api/me accepts full_name (required string, maximum 120 characters) and student_number (nullable string, maximum 30). The caller is identified from the verified Firebase token.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Profile update from the page

resources/js/pages/UserProfile.vue | source line 73 onward; blank and comment lines omitted

```
const saveProfile = async () => {
    error.value = '';
    confirmingSave.value = false;
    saving.value = true;
    try {
        const fullName = `${profile.value.firstName} ${profile.value.lastName}`.trim();
        await updateAccount({
            full_name: fullName,
            student_number: accountProfile.value?.student_number
                ?? accountProfile.value?.studentNumber
                ?? null,
        });
        applyFromAccount(accountProfile.value);
        isEditing.value = false;
        saved.value = true;
        setTimeout(() => { saved.value = false; }, 3000);
    } catch (e) {
        error.value = e?.message ?? String(e);
    } finally {
        saving.value = false;
    }
};
```

Explanation

The page combines the first and last name and sends the existing student number with the updated full_name. It waits for the account service before displaying success. saving disables repeat submission and is reset in finally, including when the API fails.

Physical Code Explanation

1 The confirmation flag is cleared before saving and the prior error is removed.

2 fullName joins the two editable name fields and trims outer whitespace.

3 updateAccount receives only the fields permitted by this screen. applyFromAccount then reloads the returned shared profile.

4 The success message clears after three seconds. catch stores the error text; finally restores the button state.

Email, role and student number are displayed as account details. In this baseline the screen edits names, although the backend also accepts a student_number value. Phone-number and email-edit functionality belongs to later code, not this Firebase profile implementation.

3 Data Connect profile persistence

app/Repositories/UserRepository.php | source line 135 onward; blank and comment lines omitted

```
public function updateProfile(string $firebaseUid, string $fullName, ?string
$studentNumber): void
    {
        $keyFirebase = 'sqlconnect_user_firebase_uid_' . $firebaseUid;
        $existing = $this->findByFirebaseUid($firebaseUid);
        Cache::forget($keyFirebase);
        if ($existing) {
            Cache::forget('sqlconnect_user_id_' . $existing->id);
        }
        $this->client->execute('UpdateUserProfile', [
            'firebaseUid'    => $firebaseUid,
            'fullName'       => $fullName,
            'studentNumber'  => $studentNumber,
        ]);
        Cache::forget($keyFirebase);
        if ($existing) {
            Cache::forget('sqlconnect_user_id_' . $existing->id);
        }
    }
}
```

Explanation

The repository resolves the current user, invalidates UID and application-ID cache entries, executes UpdateUserProfile, and invalidates again. The mutation updates fullName and studentNumber for the verified firebaseUid. The controller rereads the record so the browser receives persisted values.

Physical Code Explanation

1 The UID-specific cache key is built from the verified identity; the application ID cache is also cleared if a record exists.

2 execute passes camelCase variables to GraphQL even though the REST payload uses snake_case. This mapping is deliberate.

3 UpdateUserProfile changes only the profile fields in its mutation. It does not update Firebase Authentication credentials or promote a role.

When adding a profile field, change the schema, query selections, mutation variables, DTO conversion, request validation and Vue form together. Merely adding an input cannot make the field persist.

4 Verification and Maintenance

Verification checklist

1 Change the name, confirm Save, refresh the page and verify the value in the API response and in a booking label.

2 Cancel editing and confirm the original name is restored without a request.

3 Send a missing or over-length full_name: expect Laravel validation failure with HTTP 422.

4 Attempt to send another firebase_uid or role in the request: the update must remain scoped to the authenticated user and must not change privileges.

5 Simulate API failure and confirm the form remains recoverable and saving returns to false.

Implementation boundaries

The screen treats student number as read-only while the API permits its update. Decide whether it should be immutable and enforce that decision on the backend.

A new display name can appear in several cached or already-loaded views. Refresh the shared account and any affected booking data after changes; do not treat stale browser labels as a failed database update.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).